Physics Informed Neural Network: A Brief Review and Examples

Wongel Nadew

April 2024

1 Introduction

Physics-informed machine learning is concerned with leveraging existing knowledge to improve machine learning systems and learn new physics for complex systems that classical physics cannot explain accurately[2]. Rather than abandoning the knowledge base of physics humans accumulated over centuries, it can be used with regression and optimization techniques to help us build physical models. This enhances the model's capability to represent complex systems that classical physics cannot explain accurately. Some of the applications of physics-informed machine learning:

- Development of new materials (Solid mechanics)
- Understanding turbulence (Fluid mechanics)
- Optimizing existing designs: Wind turbine design

Physics-informed Neural Networks(PINNs) are one of the machine learning techniques that utilize Neural Networks along with physics laws constraining the neural network. Nowadays, Physics-Informed Neural Networks (PINNs) are utilized to solve Partial Differential Equations (PDEs), fractional equations, integral-differential equations, and stochastic PDEs[3]. The main advantage of PINNs is their capability to model complex phenomena with reasonable generalization. The physical constraints imposed on the neural networks could be:

- Conservation laws (Conservation of momentum, mass, energy, charge)
- Naturally occurring symmetry
- Invariants

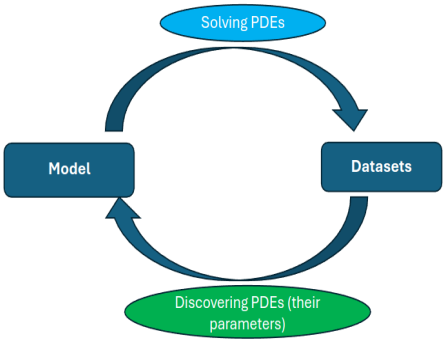
These constraints enable us to construct a model with:

- Best fit for the data available
- Generalizable (performs well even on the data it has not seen)

According to Raissi et.al[5], PINNs can be applied for two applications. These are solving initial and boundary value PDEs and discovering models from datasets.

1.1 Solving partial differential equations

Provided a model as a form of a partial differential equation (PDE), it is possible to compute the solution for the model over a range of temporal and spatial axes constrained to the initial and boundary conditions imposed. By picking random collocation points, and subjecting them to the model physics constraints, it is possible to extract solutions for the model. This method of solving PDEs does not offer substantial accuracy improvement and computation efficiency compared to finite element methods.



1.2 Discovering Models

Another interesting application of PINNs is to discover PDE models from a given dataset which is the inverse problem of solving PDEs. This topic is highly interesting and successive sections provide a walk-through on how models can be extracted from data.

2 Problem Setup

We are considering a partial differential equation (PDE) of the form:

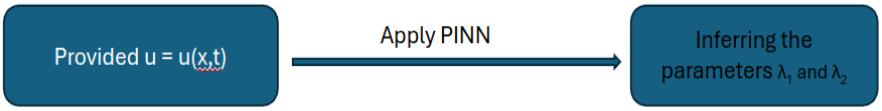
$$u_t + \mathcal{N}(u; \lambda) = 0 \tag{1}$$

where $\mathcal{N}$ is a non-linear operator. For a one-dimensional burger equation, the non-linear operator will take the form:

$$\mathcal{N}(u; \lambda) = \lambda_1 u\, u_x - \lambda_2 u_{xx} \tag{2}$$

Applying PINNs requires optimizing two components:

- The neural network
- The parameters λ_1 and λ_2



Optimizing these two components gives rise to two loss functions:

- The neural network loss
- Physical loss

3 Network Architecture

The general structure of PINNs can be considered to be a three-part modular structure comprised of:

- The neural network approximation
- The component that corrects approximation (the physics-informed network, i.e. the physics loss)
- The module that manages the minimization of the losses (optimizer)

Since we need access to the loss function to incorporate more terms in the loss function, the user defines the neural network. Therefore the weights and biases are defined along with it thus the neural net is not a black box anymore. The physics component of the system is defined as:

$$f := u_t + \mathcal{N}(u) \tag{3}$$

The loss function has two parts and it is constructed using mean-squared error.

$$\begin{aligned} \mathcal{L} &= MSE = MSE_u + MSE_f \\ MSE_u &= \sum_{i=1}^{N_u} (u(x,t) - u^*(x,t))^2 \\ MSE_f &= \sum_{i=1}^{N_u} (f(x,t))^2 \end{aligned} \tag{4}$$

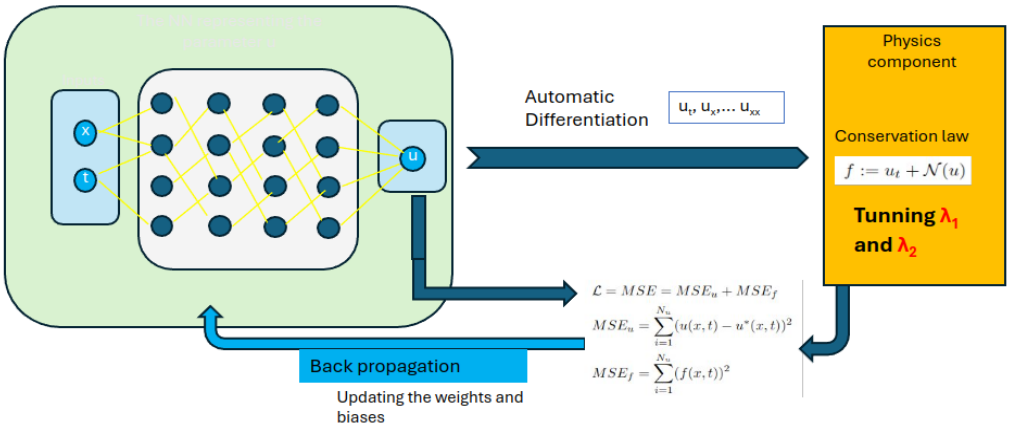


Figure 1: Typical PINN architechture

3.1 Discovering Models

4 Examples

4.1 Diffusion equation

A diffusion equation with a diffusion coefficient D = 6 is numerically solved using the Neumann boundary condition on both sides.

$$\begin{aligned} &c_t = 6\, c_{xx} \\ &c_x(0,t) = 50; \;\; c_x(100,t) = 0 \\ &c(x,0) = 0 \end{aligned} \tag{5}$$

Using only 5000 sparse collocation points from 100,000 available data points (5%), the PINN can determine the diffusion coefficient with good accuracy. The coefficient error for the diffusion coefficient (D) is 0.334 %.
Identified Equation:

$$c_t = 5.98\, c_{xx} \tag{6}$$

4.2 Dynamics: Mass Damper Spring System

The mass-spring-damper model consists of discrete mass nodes distributed throughout an object and interconnected via a network of springs and dampers. Governing equation for a mass damper spring system

$$m\,\ddot{x} + c\,\dot{x} + k\,x = F(t) \tag{7}$$

Where: m: mass
c: damping coefficient

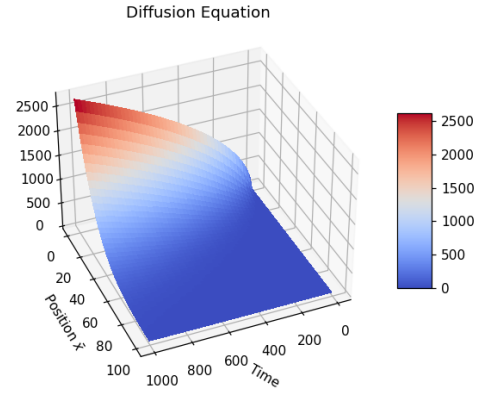


Figure 2: Concentration profile over x and t axes

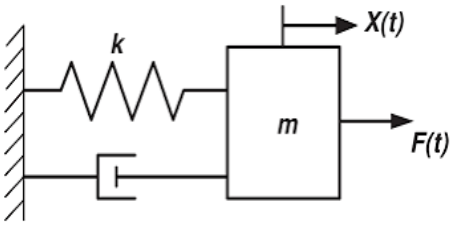


Figure 3: A mass-damper-spring system

k: spring constant

The coefficients obtained from the PINN match the original parameters very well. The coefficient errors for the three parameters are presented below.

m (λ_1)	c (λ_2)	k(λ_3)
1 %	0%	1%

4.3 Solid Mechanics: Linear Elasticity

A synthetic data from FEM solutions solved using Fenics[1] and Lame's Constants λ=1 and μ =0.5 is generated for training. A cantilever beam (2D structure) clamped at one end deforming due to its weight. A dataset is generated by extracting mesh coordinates, displacement, and stress at all the nodes. Then the data gathered is post-processed to calculate the nodal forces. Three constraints were applied to the PINN constructed to retrieve the lame parameters:

- Stress equilibrium equation
- Constitutive relation

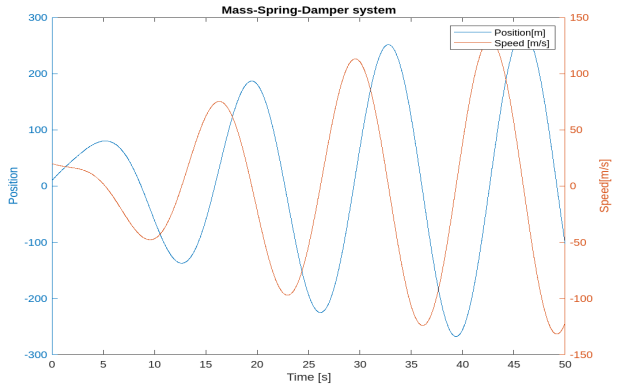


Figure 4: Solution to mass-damper-spring system

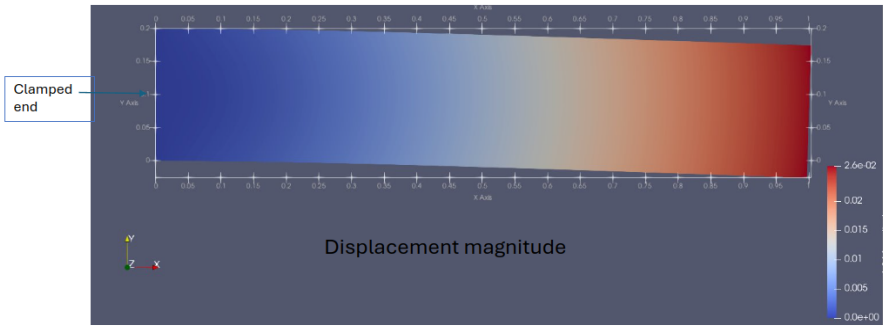


Figure 5: Displacement magnitude of a bar

- Kinematic relation

$$\begin{aligned} \sigma_{ij,j} + f_i &= 0 \\ \sigma_{ij} &= \lambda\, \delta_{ij} \epsilon_{kk} + 2\, \mu \epsilon_{ij} \\ \epsilon_{ij} &= \frac{1}{2}(u_{i,j} + u_{j,i}) \end{aligned} \tag{8}$$

Five neural networks corresponding to five important parameters, namely $u_x.u_y, \sigma_{xx}, \sigma_{xy}, \sigma_y y$ were defined separately, but their loss functions were bundled together. This method is more effective than defining only two neural networks for u_x, u_y[4]. Defining five distinct neural networks requires a very high computational capacity, so the results are not presented here.

5 More works on PINNs

5.1 PINNs for optimization

Vital tasks including designing chips, and wind turbines, finding spacecraft paths, modeling super materials, and creating reactor plans are essential for the rapidly evolving technological development. PINN uses a special function that combines conservation laws, and previously known physics and aims to search for the best solutions from a broad perspective[6]. Additional goals can be imposed to constrain the PINN to single out optimized parameters that meet certain goals defined by the user. This can be achieved by embedding the goal during loss computation.

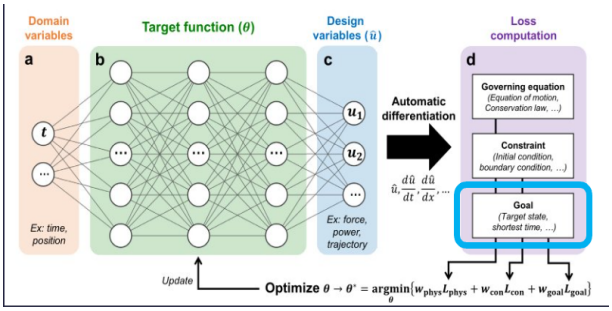


Figure 6: Integrating optimization goal into the loss function

5.2 PINNs for parsimonious model discovery

The data-driven discovery of partial differential equations (PDEs) has gained momentum recently. Provided that the model dynamics are governed by a few terms, it's possible to derive a sparse model with reasonable accuracy and good generalizability. A physics-informed information criterion (PIC) can be used to assess the simplicity and representativeness of the model during the discovery process[7]. Xu et. al. established a framework to extract a parsimonious model from data using PINNs.

6 Summary

Physics Informed Neural Networks can be applied either to solve a model and produce datasets or discover new models from data. The second application makes PINNs indispensable tools to unveil complex model dynamics that classical physics fails to work well. Neural Networks (NN) have superiority in modeling nonlinear systems compared to traditional regression techniques, making them an ideal choice for the application of discovering nonlinear dynamic systems. Incorporating previously known physics into neural networks can help

unveil models that can accurately capture complex system dynamics with good interpretability. A considerable refinement to this framework would be robustifying it to noise. Real-world data is often rife with noise, and this noise typically follows a non-Gaussian distribution further exacerbating the problem. Therefore, improving the framework's ability to manage this noise would significantly boost its performance.

References

[1] Martin S. Alnaes, Anders Logg, Kristian B. Oelgaard, Marie E. Rognes, and Garth N. Wells. Unified form language: A domain-specific language for weak formulations of partial differential equations, 2013.

[2] Steven L. Brunton. Physics informed machine learning: High-level overview of ai and ml in science and engineering.

[3] Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics–informed neural networks: Where we are and what's next. *Journal of Scientific Computing*, 92(3):88, 2022.

[4] Ehsan Haghighat, Maziar Raissi, Adrian Moure, Hector Gomez, and Ruben Juanes. A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. *Computer Methods in Applied Mechanics and Engineering*, 379:113741, 2021.

[5] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[6] Jaemin Seo. Solving real-world optimization tasks using physics-informed neural computing. *Scientific Reports*, 14(1):202, 2024.

[7] Hao Xu, Junsheng Zeng, and Dongxiao Zhang. Discovery of partial differential equations from highly noisy and sparse data with physics-informed information criterion. *Research*, 6:0147, 2023.